Aran Learn 20: Presenter Mode and a Demo That Survives Failure

For: whoever presents Aran to judges or reviewers, and whoever maintains `app/ui/js/present.js`. **Time:** about 1 hour, plus one full rehearsal. **Before this:** Learn 17 (wiring a design to a live API).

Live demos fail in boring ways: the server crashes, a model takes too long to load, the hall Wi-Fi drops. Presenter mode gives you a fixed path through the best screens and keeps the demo running when the server does not.

1. Using it

Open `http://localhost:8000/?present=1#tour/0` (or press ▶ **Present** in the top bar). A bar appears at the bottom:

Step	Screen	Time
1	Hero: a firewall for data	20 s
2	Without Aran / with Aran	30 s
3	Judges try to beat it (QR)	35 s
4	Disguises do not work	15 s
5	Two safe masks make one leak	25 s
6	The customer erases their data	25 s
7	Harvest now, decrypt later	20 s
8	The evidence	10 s

- **Next / previous:** the bar’s buttons, **PageDown / PageUp** (what a presentation clicker sends), or **N / B** on the keyboard.
- The clock counts down 3:00 from your first step and turns red when you run over.
- × leaves presenter mode.

2. Record and replay

While presenter mode is on, every successful answer from the server is saved in the browser (`localStorage['aran-recording']`), one per endpoint. The bar shows how many are saved.

If a later request fails because the server is unreachable or answers with a 5xx error, `present.js` returns the saved answer for that endpoint instead, and a yellow banner says **“Offline replay: showing the last answers recorded from the live server”**. When the server answers again, the banner goes away by itself.

This is honest on purpose: the banner is always visible during replay, and every replayed value is one the real server produced earlier.

So: rehearse once with the server on, clicking every step and every button you will use. That fills the recording.

3. How it works

Wrapping a function

Every page calls the server through `A.api` (Learn 17). `present.js` wraps it:

```
        const live = A.api
        A.api = async (path, body, method) => {
          try { const d = await live(path, body, method); rec[key] = d;
save(); return d }
          catch (e) { if (offline(e) && key in rec) { banner(true); return
rec[key] } throw e }
        }
```

No page had to change. This pattern (replace a function with one that calls the original and adds behaviour) is called a **wrapper** or **decorator**.

localStorage

`localStorage` stores text by key, about 5 MB per site, and it survives reloads. We store JSON with `JSON.stringify`, skip answers over 400 KB, and wrap every write in `try` because it can be full or blocked.

A service worker for the page itself

Replaying API answers is not enough if the **page** cannot load. A **service worker** (`app/ui/sw.js`) is a small script the browser runs between the page and the network. In presenter mode it keeps a copy of every console file it

fetches. Its rule is **network first**: when the server answers, its file wins and the copy is refreshed; when it does not, the copy is served. So the console still opens after the server stops.

Service workers, like the microphone (Learn 18), only run in a secure context: `https://` or `localhost`. Present from the laptop that runs the server.

4. Demo checklist

1. The day before: run the tests (`python -m pytest -q` in `research` and `app/server`), and `node app/ui/tools/check-i18n.js`.
2. One hour before: start the server, open presenter mode, click through every step once (this records the answers).
3. Teach the immune layer (`#immune`, “Learn normal traffic”) at least 6 minutes before you need its alerts.
4. For the arena: start with `--host 0.0.0.0`, connect laptop and judges to your phone hotspot, scan the QR yourself first.
5. Sound on, brightness up, notifications off.

Practice

1. Rehearse once, then stop the server and click through again. Find which screens replay and which do not (a button you never pressed during rehearsal has nothing saved).
2. Open the browser’s DevTools, Application tab, and look at the recording and the service worker.
3. Time yourself on the full path. Cut words until you finish in 3:00.